

Nível de Microarquitetura (parte 4)



UNIFEI

Universidade Federal de Itajubá

IMC – Instituto de Matemática e Computação

Prof. Carlos Minoru Tamaki

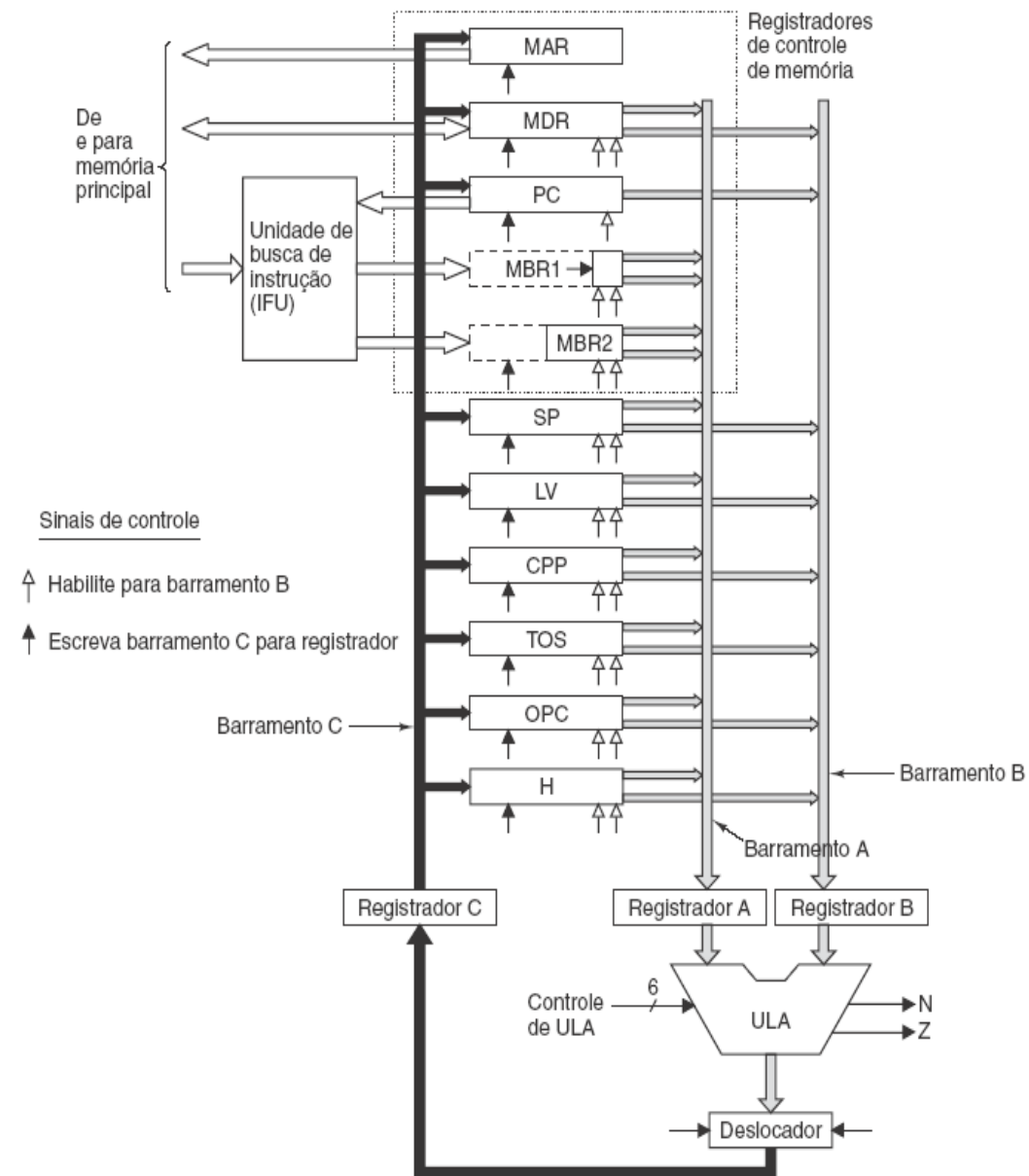
Projeto com paralelismo: a Mic-3

O ciclo de relógio é limitado pelo tempo necessário para que os sinais se propaguem pelo caminho de dados. Há três componentes importantes no ciclo do caminho de dados propriamente dito:

1. O tempo para levar os registradores selecionados até os barramentos A e B.
2. O tempo para que a ULA e o deslocador realizem seu trabalho.
3. O tempo para os resultados voltarem aos registradores e serem armazenados.

Arquitetura de três barramentos

Mostramos uma nova arquitetura de três barramentos, incluindo a IFU, mas com três registradores adicionais, cada um inserido no meio de cada barramento. Os registradores são escritos em todo ciclo. Na realidade, os registradores repartem o caminho de dados em partes distintas que agora podem funcionar independentemente uma da outra. Denominaremos isso Mic-3, ou modelo com paralelismo (ou pipelined).



- Como esses registradores extras podem ajudar? Agora leva três ciclos de relógio para usar o caminho de dados:
 - um para carregar os registradores A e B,
 - um para executar a ULA e deslocador e
 - carregar o registrador C e
 - um para armazenar o serializador C de volta nos registradores.
- Estamos loucos? (Dica: Não!) Inserir os registradores têm dupla finalidade:
 1. Podemos aumentar a velocidade do relógio porque o atraso máximo agora é mais curto.
 2. Podemos usar todas as partes do caminho de dados durante todo ciclo.

Implementação de SWAP

Rótulo	Operações	Comentários
swap1	MAR = SP - 1; rd	Leia a 2a palavra da pilha; ajuste MAR a SP
swap2	MAR = SP	Prepare para escrever nova 2a palavra
swap3	H = MDR; wr	Salve novo TOS; escreva 2a palavra para pilha
swap4	MDR = TOS	Copie TOS antigo para MDR
swap5	MAR = SP - 1; wr	Escreva TOS antigo para o 2o lugar na pilha
swap6	TOS = H; goto (MBR1)	Atualize TOS

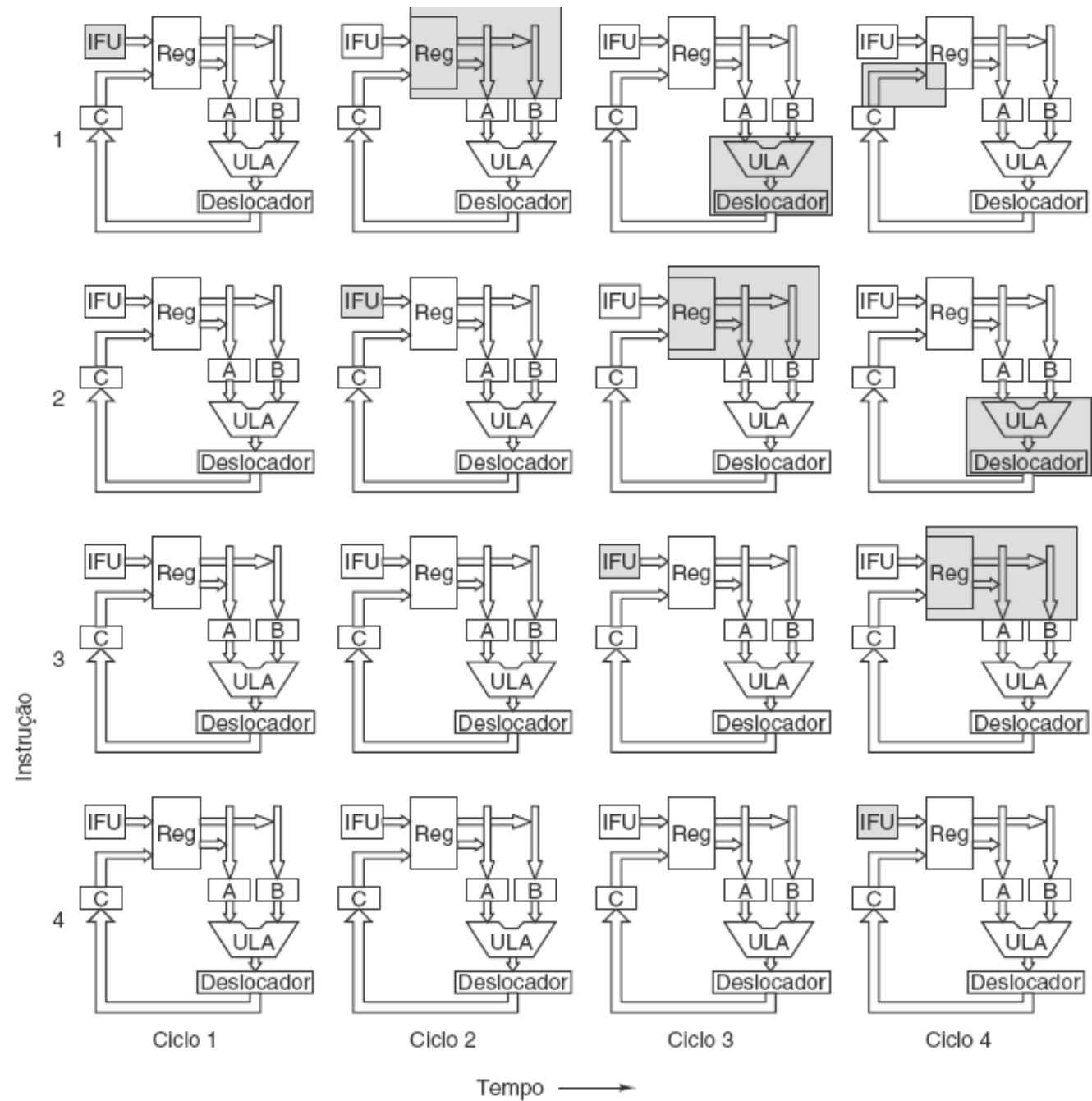
Código da Mic-2 para SWAP

	Swap1	Swap2	Swap3	Swap4	Swap5	Swap6
Ci	MAR=SP- 1;rd	MAR=SP	H=MDR,wr	MDR=TOS	MAR=SP- 1;wr	TOS=H;goto (MBR1)
1	B=SP					
2	C=B-1	B=SP				
3	MAR=C; rd	C=B				
4	MDR=Mem	MAR=C				
5			B=MDR			
6			C=B	B=TOS		
7			H=C; wr	C=B	B=SP	
8			Mem=MDR	MDR=C	C=B- 1	B=H
9					MAR=C; wr	C=B
10					Mem=MDR	TOS=C
11						goto (MBR1

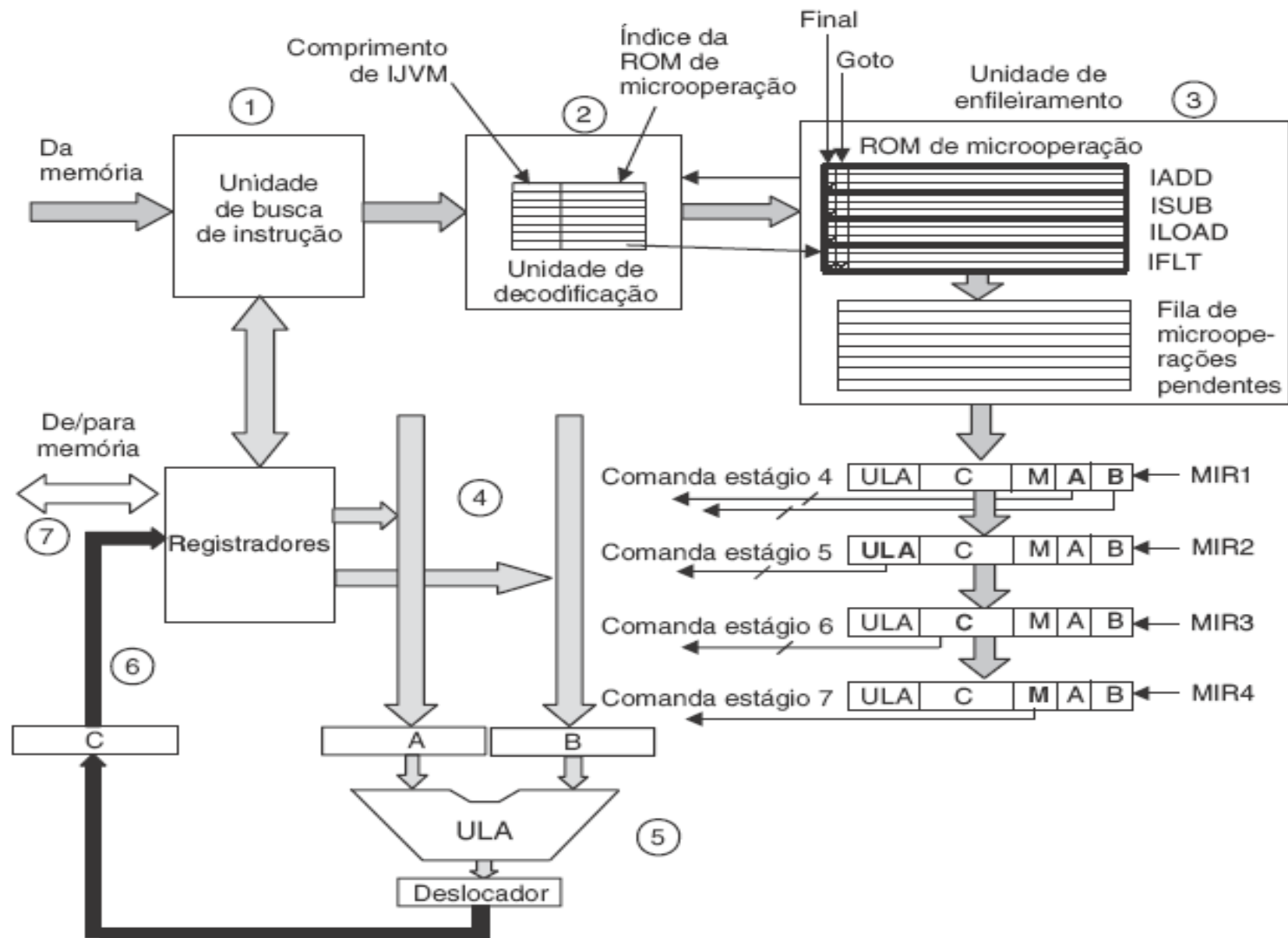
Implementação de SWAP na Mic-3

Paralelismo

Ilustração gráfica
do funcionamento
de paralelismo
(pipeline).

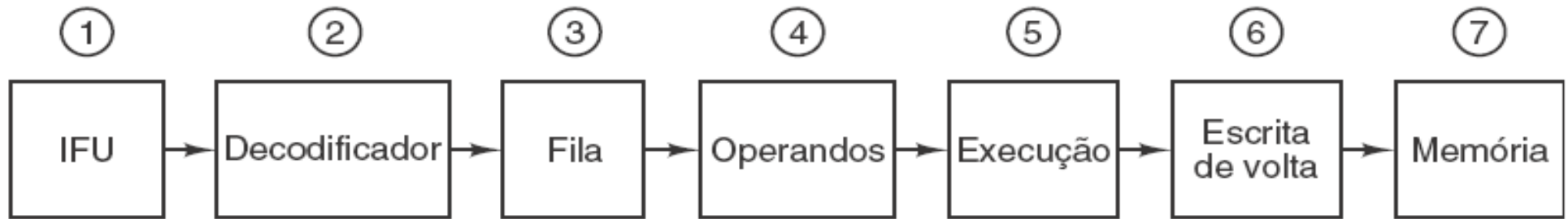


Paralelismo de sete estágios: a Mic-4



Principais componentes da Mic-4

Paralelismo da Mic-4



Quando Edsger Dijkstra escreveu seu famoso artigo "*GOTO statement considered harmful*" (Declaração GOTO considerada perigosa), ele não tinha idéia do quanto estava certo.

A Mic-1 era uma peça de hardware muito simples, com quase todo o controle em software. A Mic-4 tem um projeto de alto paralelismo, com sete estágios e hardware muito mais complexo. A Mic-4 faz busca antecipada automática de uma sequência de bytes da memória, decodifica-a para instruções IJVM, converte-a para uma sequência de microoperações usando uma ROM e a enfileira para usar quando necessário. Os primeiros três estágios do paralelismo podem ser vinculados ao relógio do caminho de dados se desejado, mas nem sempre haverá trabalho a fazer.

Melhoria de desempenho

Todos os fabricantes de computadores querem que seus sistemas funcionem o mais rapidamente possível.

Veremos algumas técnicas avançadas que estão sendo investigadas para melhorar o desempenho do sistema (CPU e memória). Devido à natureza de alta competitividade da indústria de computadores, a defasagem entre novas ideias que podem tornar um computador mais rápido e sua incorporação a produtos é surpreendentemente rápida. Por consequência, a maioria das ideias que discutiremos já está em uso em uma vasta maioria de produtos existentes.

As ideias que discutiremos podem ser classificadas em duas grandes categorias:

- Melhorias de implementação são modos de construir uma nova CPU ou memória para fazer o sistema funcionar mais rapidamente sem mudar a arquitetura. Modificar a implementação sem alterar a arquitetura significa que programas antigos serão executados na nova máquina, um importante argumento de venda. Um modo de melhorar a implementação é usar um relógio mais rápido, mas não é o único. Os ganhos de desempenho obtidos na família 80386, 80486, Pentium, Pentium Pro até o Pentium II se devem a implementações melhores, porque, em essência, a arquitetura permaneceu a mesma em todas elas.

- Alguns tipos de melhoria só podem ser feitos com a alteração da arquitetura. Às vezes essas alterações são incrementais, como adicionar novas instruções ou registradores, de modo que programas antigos continuarão a ser executados nos novos modelos. Nesse caso, para conseguir um desempenho completo, o software tem de ser alterado, ou ao menos recompilado com um novo compilador que aproveita as novas características. Contudo, passadas algumas décadas, os projetistas percebem que a antiga arquitetura, durou mais do que sua utilidade e que o único modo de progredir é começar tudo de novo. A revolução RISC na década de 1980 foi uma dessas inovações, uma outra está no ar agora. Vamos examinar um exemplo o Intel IA-64.

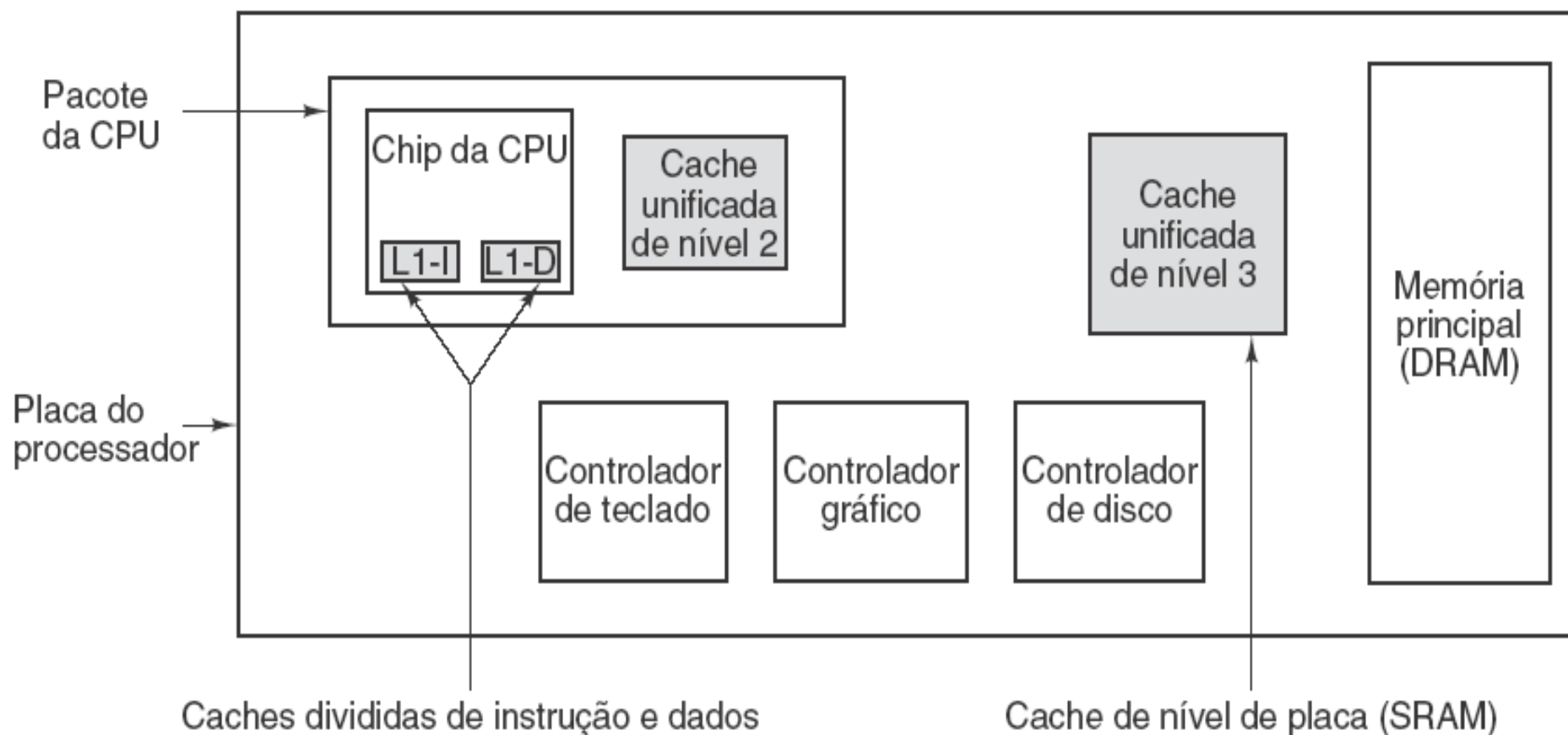
Memória cache

Um dos aspectos mais desafiadores do projeto de um computador em toda a história tem sido oferecer um sistema de memória capaz de fornecer operandos ao processador à velocidade em que ele pode processá-los. A recente alta taxa de crescimento na velocidade do processador não foi acompanhada de um aumento correspondente na velocidade das memórias. Se comparadas com as CPUs, as memórias estão ficando mais lentas há décadas. Dada a enorme importância da memória primária, essa situação limitou muito o desenvolvimento de sistemas de alto desempenho e estimulou a pesquisa a encontrar maneiras de contornar o problema da velocidade das memórias que são muito menores do que as velocidades das CPUs e, em termos relativos, estão ficando piores ano após ano.

Processadores modernos exigem muito de um sistema de memória, tanto em termos de latência (o atraso na entrega de um operando) quanto de largura de banda (a quantidade de dados fornecida por unidade de tempo). Infelizmente, há um grande antagonismo entre esses dois aspectos de um sistema de memória. Muitas técnicas para aumentar a largura de banda também aumentam a latência. Por exemplo, as técnicas de paralelismo usadas na Mic-3 podem ser aplicadas a um sistema de memória que tenha várias memórias sobrepostas e elas serão manipuladas com eficiência. Infelizmente, assim como na Mic-3, isso resulta em maior latência para operações individuais de memória. À medida que aumentam as velocidades de relógio do processador, fica cada vez mais difícil prover um sistema de memória capaz de fornecer operandos em um ou dois ciclos de relógio.

Um modo de atacar esse problema é providenciar caches. Uma cache guarda as palavras de memória usadas mais recentemente em uma pequena memória rápida, o que acelera o acesso a elas. Se uma porcentagem suficientemente grande das palavras de memória estiver na cache, a latência efetiva de memória pode ter enorme redução.

Memória cache



Sistema com três níveis de cache.

Caches dependem de dois tipos de endereço de localidade para cumprir seu objetivo:

- **Localidade espacial** é a observação de que localizações de memória com endereços numericamente similares a uma localização de memória recentemente acessada provavelmente serão acessadas no futuro próximo. Caches exploram essa propriedade trazendo mais dados do que os requisitados, na expectativa de poder antecipar requisições futuras.
- **Localidade temporal** ocorre quando localizações de memória recentemente acessadas são acessadas novamente. Isso pode ocorrer, por exemplo, com localizações de memórias próximas ao topo da pilha, ou com instruções dentro de um laço. Localidade temporal é explorada em projetos de cache primariamente pela da escolha do que descartar quando ocorre uma ausência na cache. Muitos algoritmos de substituição de cache exploram localidade temporal descartando as entradas que não foram acessadas recentemente.

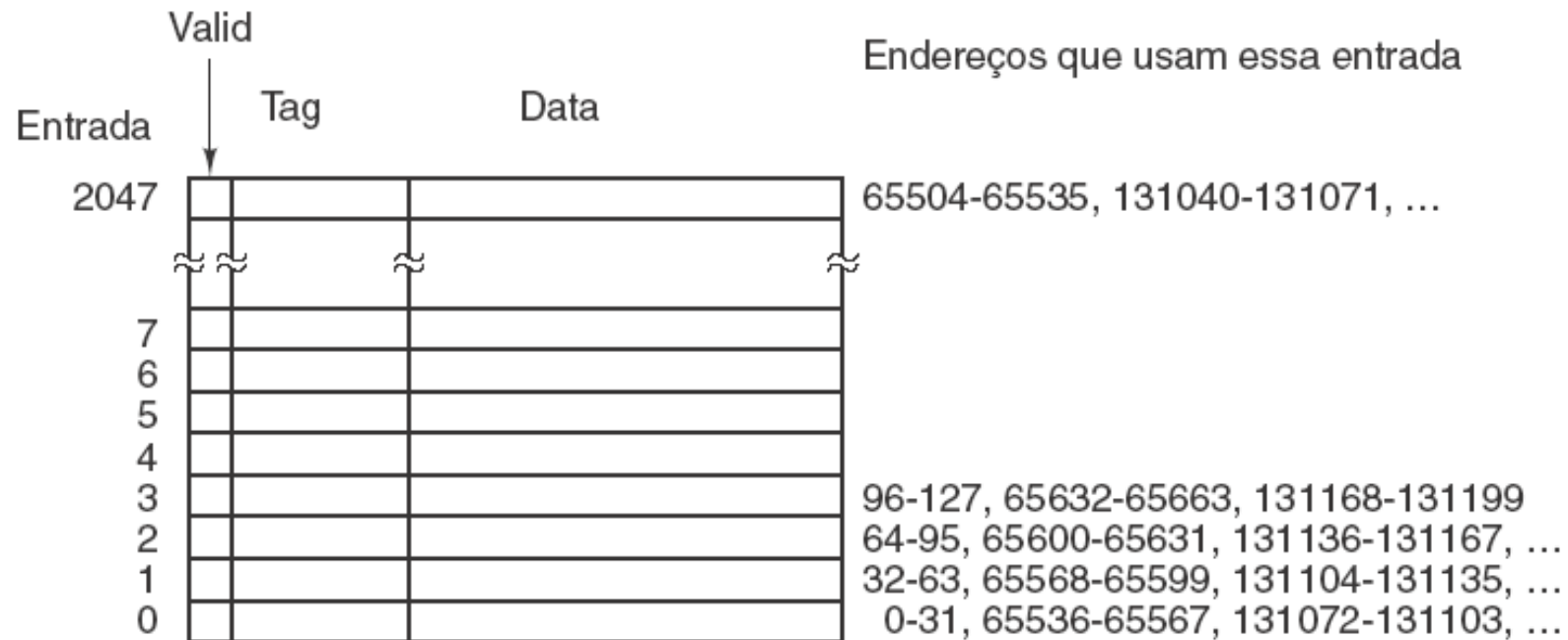
- Cada entrada de cache consiste em três partes:
 1. O bit Valid indica se há ou não quaisquer dados válidos nessa entrada. Quando o sistema é iniciado, todas as entradas são marcadas como válidas.
 2. O campo Tag consiste em um único valor de 16 bits que identifica a linha de memória correspondente da qual vieram os dados.
 3. O campo Data contém uma cópia dos dados na memória. Esse campo contém uma linha de cache de 32 bytes.

Caches de mapeamento direto

Em uma cache de mapeamento direto, uma determinada palavra de memória pode ser armazenada em exatamente um lugar dentro da cache. Dado um endereço de memória, há somente um lugar onde procurar por ele na cache. Se ele não estiver nesse lugar, então não está na cache. Para armazenar e recuperar dados da cache, o endereço é desmembrado em quatro componentes, como mostrado a figura anterior:

1. O campo TAG corresponde aos bits TAG armazenados em uma entrada de cache.
2. O campo LINHA indica qual entrada de cache contém os dados correspondentes, se eles estiverem presentes.
3. O campo WORD informa qual palavra dentro de uma linha é referenciada.
4. O campo BYTE em geral não é usado, mas se for requisitado apenas um byte, ele informa qual byte dentro da palavra é necessário. Para uma cache que fornece apenas palavras de 32 bits esse campo será sempre 0.

Caches de mapeamento direto



(a)



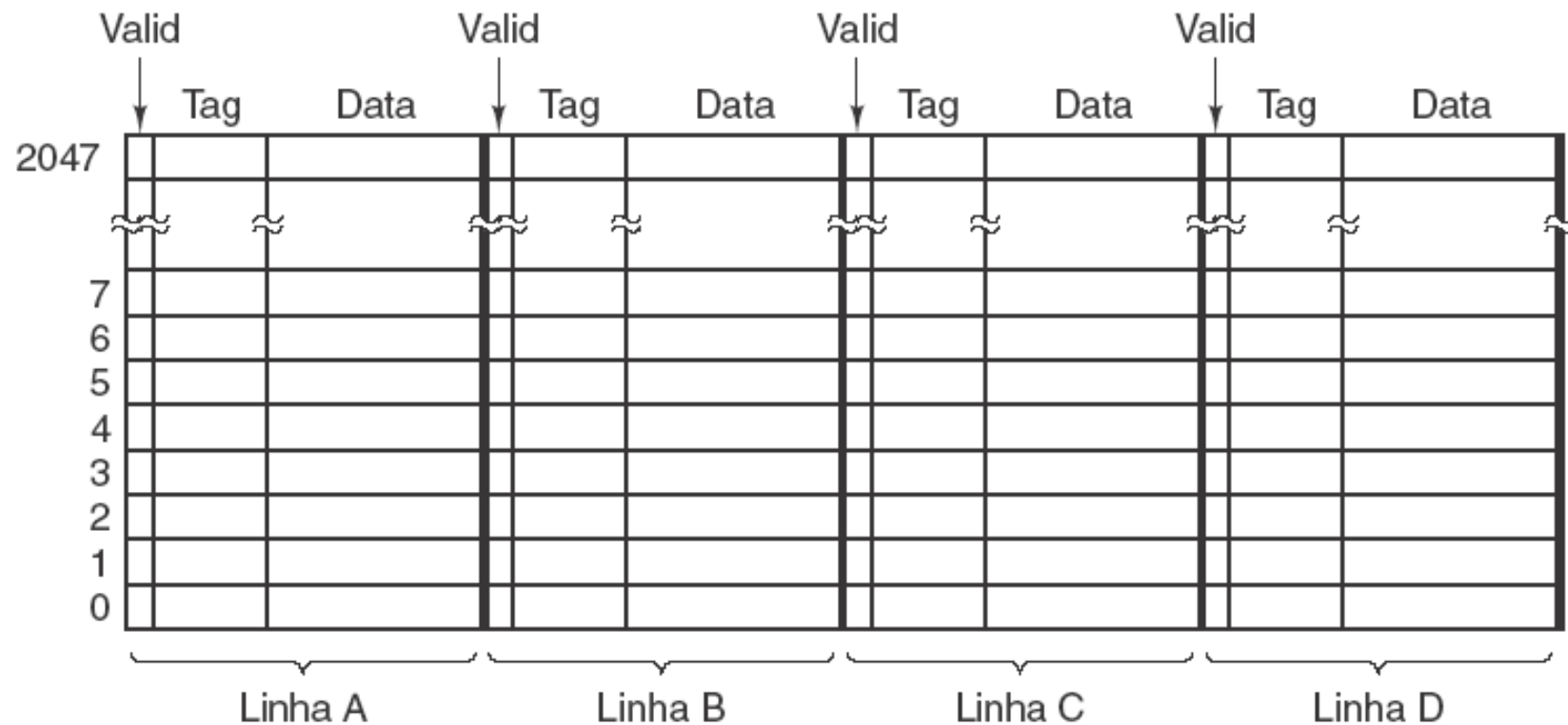
(b)

(a) Cache de mapeamento direto. (b) Endereço virtual de 32 bits.

Caches associativas de conjunto

- Como mencionamos antes, muitas linhas diferentes competem na memória pelas mesmas posições na cache (cache slots). Se um programa que utiliza a cache de armazenamento direto usar muito palavras nos endereços 0 e 65.536, haverá conflitos constantes porque cada referencia potencialmente expulsaria a outra da cache. Uma solução para este problema é permitir duas ou mais linhas em cada entrada de cache. Uma cache com n entradas possíveis para cada endereço é denominada uma **cache associativa de conjunto de n vias**. Uma cache associativa de conjunto de quatro vias é ilustrada na próxima figura.

Caches associativas de conjunto



Cache associativa de conjunto de 4 vias.

- Por fim, escritas propõem um problema especial para as caches. Quando um processador escreve uma palavra e a palavra está na cache, é óbvio que ele tem de atualizar a palavra ou descartar a entrada da cache. Praticamente todos os modelos atualizam a cache. Mas, e quanto a atualizar a cópia na memória principal? Essa operação pode ser adiada até mais tarde, quando a linha de cache estiver pronta para ser substituída pelo algoritmo LRU. Essa escolha é difícil, e nenhuma das opções é claramente preferível. A atualização imediata da entrada na memória principal é denominada **escrita direta (write through)**. Essa abordagem geralmente é mais simples de implementar e mais confiável, uma vez que a memória está sempre atualizada - é útil, por exemplo, se ocorrer um erro e for necessário recuperar o estado da memória. Infelizmente, também requer mais tráfego de escrita para a memória, portanto implementações mais sofisticadas tendem a empregar a alternativa conhecida como **escrita retardada (write deferred)** ou **escrita retroativa (write back)**.

Predição de desvio

```
if (i == 0)
    k = 1;
else
    k = 2;
```

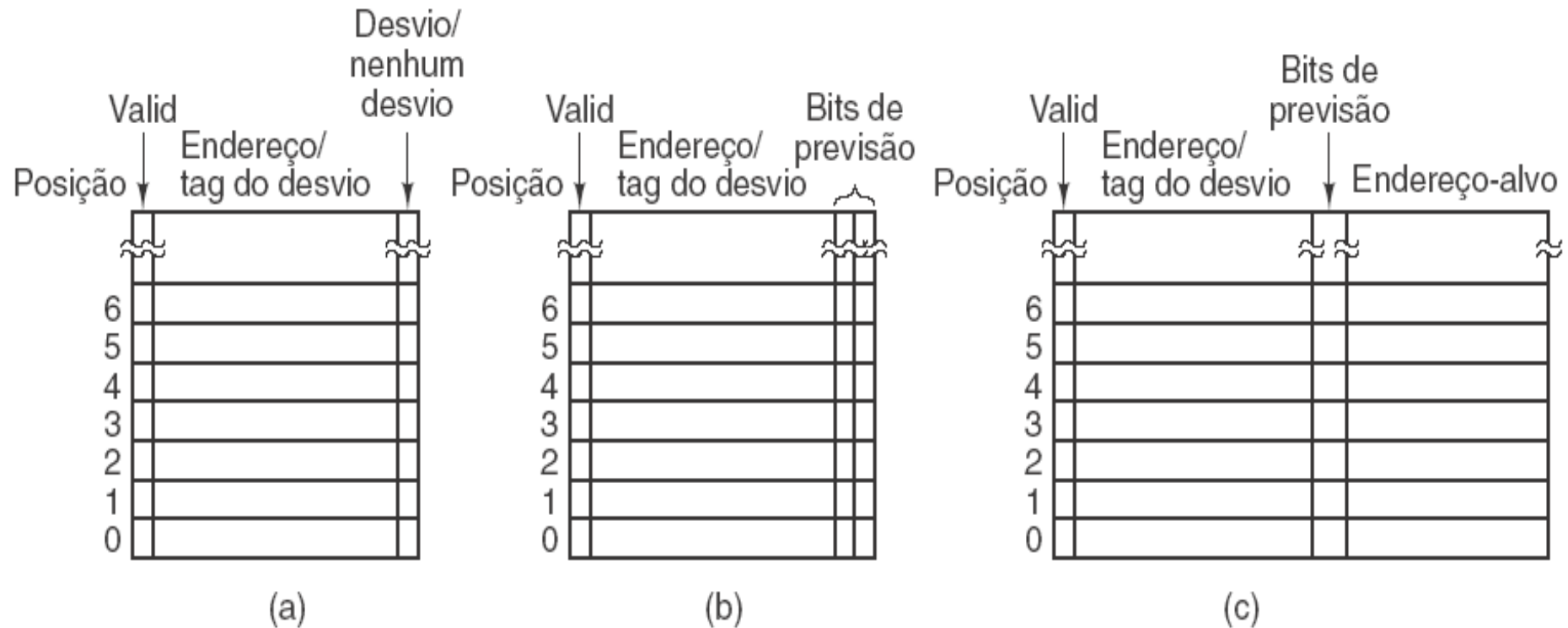
(a)

```
                CMP i,0    ; compare i com 0
                BNE Else   ; desvie para Else se não for igual
Then:           MOV k,1    ; mova 1 para k
                BR Next    ; desvio incondicional para Next
Else:           MOV k,2    ; mova 2 para k
Next:
```

(b)

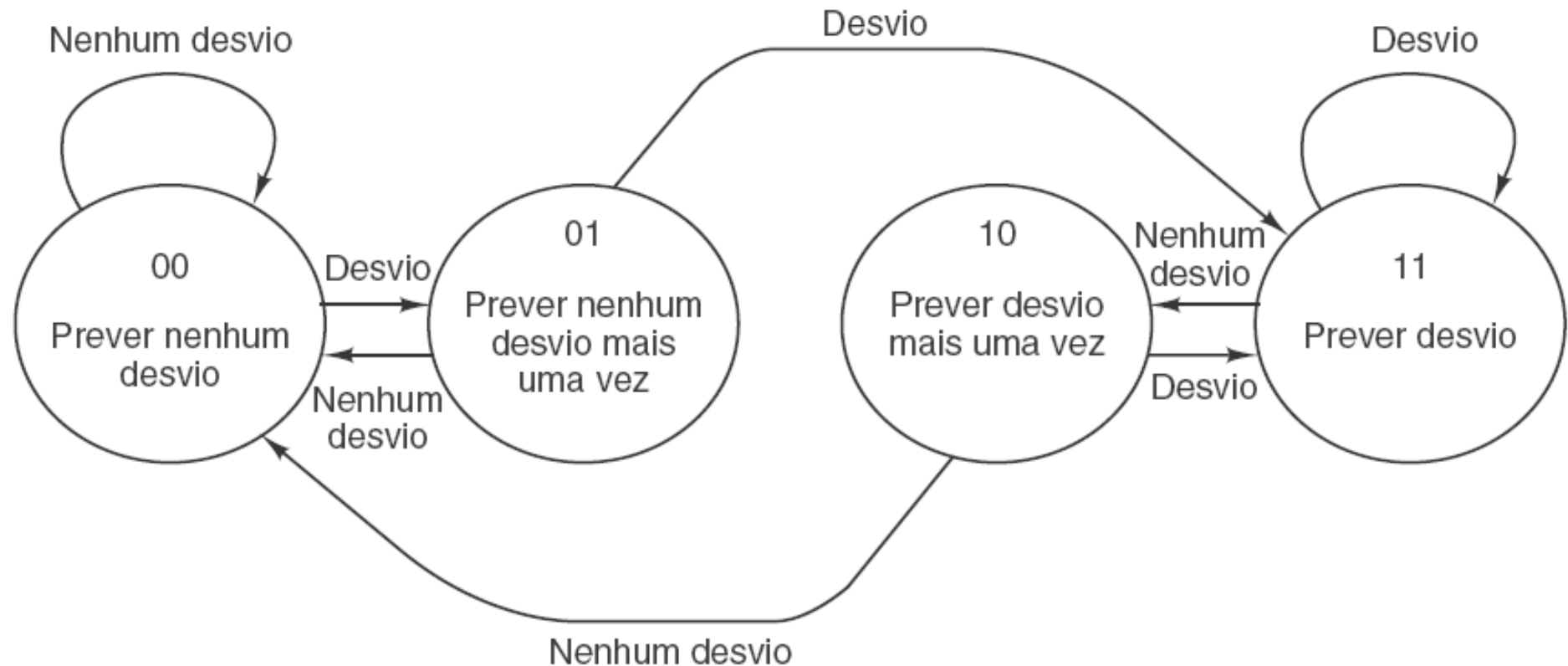
(a) Fragmento de programa.
(b) Sua tradução para uma linguagem de montagem genérica.

Previsão dinâmica de desvio



- (a) Histórico de desvio de 1 bit. (b) Histórico de desvio de 2 bits.
(c) Mapeamento entre endereço de instrução de desvio e endereço-alvo.**

Previsão dinâmica de desvio



Máquina de estado finito de 2 bits para previsão de desvio.

Previsão estática de desvio

Declarações do tipo:

```
for ( i = 0; i < 1000000; i++ )  
{  
    ...  
}
```

Bibliografia

- Organização Estruturada de Computadores – Andrew S. Tanenbaum, Tood Austin – 6ª Edição 2013 Prentice Hall